

Manual de Utilização: Heraclitus-Forge

Modelagem, Testes e Orquestração de Dados para
HeraclitusDB

Plataformas: Compatível com GNU/Linux e Microsoft Windows

Repositório: github.com/JoseRFJuniorLLMs/Heraclitus-Forge

1. Introdução ao Heraclitus-Forge

O **Heraclitus-Forge** é a ferramenta central de modelagem e orquestração (a "forja") do ecossistema Heraclitus. Ele permite que os usuários modelem schemas complexos, gerem sementes de dados (seeds), realizem inventários e operem pipelines de ingestão que se comunicam diretamente com o backend imutável (HeraclitusDB) via gRPC.

1.1. Estrutura do Repositório

O repositório do Forge está organizado nas seguintes pastas e arquivos principais:

- **registry/**: Contém a taxonomia, ontologias e modelos de comportamento padronizados em formato HCX (Heraclitus Configuration eXchange). Ex: `linux_sshd`, `nginx_access`, `postgresql`, `windows_security`.
- **rust/**: Código-fonte em Rust para validações de alta performance e interação com esquemas de segurança.
- **samples/**: Exemplos de logs reais para testes de ingestão (ex: `fake_sshd.log`, `nginx_access.log`).
- **Arquivos Python na raiz**: Scripts operacionais como `bridge.py`, `cke_forge_pipeline.py`, `compilar_sementes.py`, `forge_compiler.py`, `forge_seeds.py`, `inserir.py` e a interface de testes web `web_forge.py`.

2. Instalação e Preparação do Ambiente

Para operar o Forge, você precisará de um ambiente Python 3 configurado (recomenda-se o uso de *virtualenvs*).

```
git clone https://github.com/JoseRFJuniorLLMs/Heraclitus-Forge.git
cd Heraclitus-Forge
python -m venv venv
# Linux:
source venv/bin/activate
# Windows:
. env\Scripts ctivate

# Instalação das dependências do Forge
pip install -r requirements.txt
```

Nota: Caso você vá utilizar os componentes de IA ou o Dashboard integrados ao Forge, instale também requirements-ai.txt e requirements-dashboard.txt.

3. Utilizando o Registry (Modelos HCX)

O **Registry** é o coração analítico do Forge. Ele armazena pacotes .hcx que definem a arquitetura, ontologia e o modelo comportamental de diferentes fontes de dados.

Cada pasta HCX (ex: registry/linux_sshd/v1.0.0.hcx/) contém:

- **architecture.yaml**: Topologia e requisitos estruturais.
- **ontology.yaml**: Definição de entidades e relacionamentos (o que é um Usuário, um IP, um Evento de Login).
- **behavior.model**: Padrões esperados e regras de detecção de anomalias (usado pelos parsers de IA/determinísticos).
- **reasoning.yaml**: Lógica de inferência para o Cérebro do sistema.
- **test_matrix.json** & **benchmarks.json**: Matrizes para validação de qualidade (QA) do parser.

4. Operações Principais (CLI Python)

4.1. Compilação de Modelos

Se você alterar ou criar novos pacotes .hcx no *registry*, você deve usar o compilador do Forge para validar a integridade dos YAMLS e gerar os stubs binários.

```
python forge_compiler.py --registry ./registry --output ./build
```

4.2. Ingestão de Dados (Pipeline)

Para ler logs estruturados ou arquivos da pasta *samples/* e enviá-los (Append) de forma segura para o HeraclitusDB, utilize os conectores do Forge.

```
# Configure a URI do banco antes de rodar
export HERACLITUS_GRPC_URL="http://localhost:7474"

python inserir.py --file samples/nginx_access.log --schema nginx_access
# ou utilizando o pipeline avançado CKE
python cke_forge_pipeline.py --source samples/postgresql.log
```

4.3. Geração de Dados de Teste (Seeds)

O Forge permite a geração de grafos sintéticos complexos (casos de uso) para testar a escalabilidade do HeraclitusDB ou calibrar algoritmos de inteligência.

```
# Gera sementes simulando cenários complexos
python forge_seeds.py --scenario "fraude_licitatoria" --scale 1000

# Compila as sementes em payloads otimizados para inserção em massa
python compilar_sementes.py
```

5. Ferramentas Adicionais

O repositório oferece utilitários que facilitam o dia a dia do engenheiro de dados.

- **web_forge.py:** Sobe um mini-servidor web local com uma interface gráfica simples para testar a criação de entidades e o status da conexão gRPC com o HeraclitusDB de maneira visual. (Execute `python web_forge.py` e acesse `http://localhost:5000`).
- **tools/forge_inventory.py:** Realiza uma auditoria no diretório do projeto, listando os modelos ativos e validando inconsistências entre as ontologias do *registry*.
- **Assinatura Criptográfica:** O arquivo `forge_sign.py` assina criptograficamente os modelos gerados antes de enviá-los à produção, assegurando a proveniência dos esquemas interpretativos.

Heraclitus-Forge - Documentação de Operação.
Consulte os diretórios `tests/` para exemplos em código (pytest) de cada módulo.